Parcel 1

The Whispering Fireflies

From three fireflies at a forest plant to a sword in the player's hand.



The selected rock in front of the statue after the fireflies arrive. Captured in Unity Play Mode.

This report follows the puzzle in the same order a player encounters it. It explains how the scene was assembled, which Animator assets play each motion, how the scripts exchange information, and where to change settings in the Inspector.

> **Project snapshot** Unity 6000.4.5f1 · `Assets/Glowkeeper_Update.unity`
> Includes the latest volume-gated humming objective and the fitted right-hand sword grip.

Use the contents on the next page to jump to a mechanic. The appendix contains the full text of the nine puzzle C# scripts from the project.

PARCEL 1 / 01

Contents and build history

Section	Page
Scene objects and input	03
Three independent firefly animations	04
The player's spell and strike clips	05
Press E at the plant	06
Sound and the detector meter	07
Press E at the stone	08
Left-click and hit timing	09
Breaking the stone and dropping the sword	10
Fitting the sword in the palm	11
Tap to equip, hold to store	12
Mission stages, text, and chimes	13
Inspector wiring and project files	14
Testing and practical limits	15
Appendix · complete puzzle C# source	16

How the puzzle evolved

Flight and spells. You supplied the existing firefly asset and avatar, asked for Animator-driven flight, and supplied the one-handed attraction and two-handed release FBXs. The project uses three looping position clips, then moves their shared anchor for each spell.

Stone and sword. You chose the rock in front of the statue, imported the low-poly sword pack, and supplied the club combo. You asked for left-click to work anywhere, edited the sword's size and placement, and asked for the handle to fit the palm.

Mission presentation. You provided the quest text and medieval HUD direction. The latest revision uses actual buzz volume to decide when the top-left humming objectives are visible.

PARCEL 1 / 02

Scene objects and input

The puzzle joins existing assets through Inspector references and two Unity Input System actions.

```
Glowkeeper_Update
  BeeSwarm [FireflyHandAttraction, FireflyStoneDetector, detector AudioSource]
    Firefly_1 / Firefly_2 / Firefly_3 [one Animator each]
      Firefly_Fred [model]; Bone.003 holds TailGlow and the visible tail tip
  FireflyPlantBuzz [stationary AudioSource]
  PlayerRoot [PlayerSwordPuzzle]
    StoredBlades [inventory storage parent]
    Axe Warrior Avatar [Animator, StoneStrikeEvents, SwordHandGrip]
      ... / Hand.R
        BeeHandTarget
        SwordHandTarget
  MagicStone [original rock, MeshCollider, LODGroup, CrackableMagicStone]
    StoneBeeTarget
    MagicStoneBlueGlow [MagicStoneGlow, overlay renderer and blue light]
    CrackStage1 / CrackStage2 / BrokenStoneRubble
  FrostSwordTreasure [FloatingSword, Rigidbody, BoxCollider]
    Sword15_Frost [visual prefab instance]
    FrostSwordLight
  Whispering Fireflies HUD [Canvas, Canvas Scaler, FirstPuzzleMission]
    Quest Objective / Poetic Subtitle / Interaction Hint / Whisper of the Swarm
```

What Unity reads when you press a button

Action in PlayerControls	Binding	Use
`Player/Interact`	`<Keyboard>/e`	Attract, release, equip, or start a storage hold. The current situation determines which operation is available.
`Player/Attack`	`<Mouse>/leftButton`	Start or restart the avatar's one-handed attack.

Both actions are Button actions. `WasPressedThisFrame()` detects the new press once. `IsPressed()` is checked during the storage hold. The HUD reads gameplay state in `LateUpdate()` and never reads the keyboard or mouse.

> The main scene already has the references assigned. The project uses Unity Input System 1.19.0, Universal Render Pipeline 17.4.0, and uGUI 2.0.0 with TextMeshPro for the mission text.

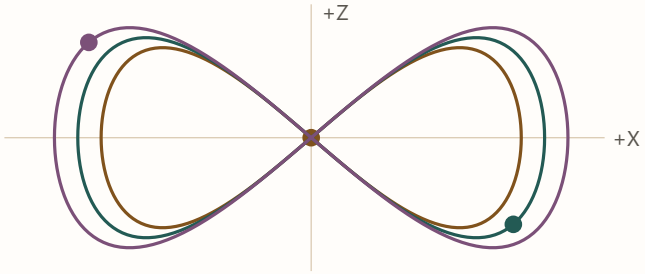
PARCEL 1 / 03

How the fireflies fly

Three Animators move the models around the plant. A shared scene object gives all three paths a new center during the spells.

Each holder under `BeeSwarm` contains an instance of the existing `Firefly_Fred` asset. Each holder's controller starts in **Figure Eight Flight**. Its matching 6-second clip loops and keys only the child model's local position, with 97 keys on each axis. The shape, rotation, scale, and model components remain available independently of the path.

TOP VIEW / LOCAL X AND Z



- Firefly 1 start phase 0°
- Firefly 2 start phase 120°
- Firefly 3 start phase 240°

Vertical bobbing is outside this view.

A useful way to describe the curve is x = A × sin(θ) and z = B × sin(2θ). The Z axis changes twice per X cycle, which creates the crossover. Y adds a small vertical bob. The scene uses baked Animator curves for this shape.

Holder	Start	State speed	X / Z radius	Local Y range
Firefly_1	0°	1.00×	0.63 / 0.27	0.05–0.15
Firefly_2	120°	1.09×	0.70 / 0.30	0.20–0.30
Firefly_3	240°	1.18×	0.77 / 0.33	0.35–0.45

Editor setup

01 Select a firefly holder and open **Window** → **Animation** → **Animator**. Select **Figure Eight Flight** to change its speed.

02 Open the Animation window and edit the matching position curves. Keep Loop Time on and match the first and last values and tangents for a clean loop.

03 Leave the flight Animators at **Always Animate** and **Normal** update mode. The travel script uses their fresh local positions each frame.

Each `Bone.003` tail has a `TailGlow` point light, set to intensity 1.5 and range 1.4, plus a small emissive tip. Both follow the insect through the hierarchy.

PARCEL 1 / 04

How the avatar animates

The existing controller layers spells and strikes over the player's normal locomotion.

Supplied FBX	Animator state	Length and start rule
Standing 1H Magic Attack 01.fbx	Attract Fireflies	2.30 s. A successful E interaction at the plant sets the `AttractFireflies` trigger.
Standing 2H Magic Area Attack 02.fbx	Release Fireflies	3.27 s. A successful E interaction at the stone sets `ReleaseFireflies`.
One Hand Club Combo.fbx	Strike Magic Stone	3.63 s. Left-click plays the state directly from the beginning.

Import and layer setup

01 Import the three supplied FBXs under `Assets/GlowKeeper/Animation/AxeWarrior`. In the Rig tab use **Humanoid**. Unity retargets the clips to the avatar's humanoid bones.

02 In the Animation tab name the clips as shown above. They are 30 FPS, have Loop Time off, and bake root rotation and position into the pose. The scene Animator has Apply Root Motion off, so movement remains controlled by the existing player controller.

03 In `AxeWarriorAnimator.controller`, use the **Firefly Casting** override layer at weight 1 with `FireflyCastingUpperBody.mask`. The mask includes body, head, arms, and fingers. The base layer still runs the walking, running, jumping, and swimming states.

04 Set **Ready** as the default state. The three action states have Write Defaults enabled and exit back to Ready at normalized time 1, with a 0.20-second transition.

Transitions and the two ways to start an animation

Spell triggers. `Ready → Attract Fireflies` uses a 0.12-second blend. `Any State → Release Fireflies` also blends for 0.12 seconds and can interrupt attraction. Both spells start travel immediately when their triggers are set.

Mouse click. The controller contains a `StrikeStone` trigger transition, but the current input script uses `Animator.Play(..., 0f)` on the casting layer. That is what makes another click restart the combo immediately.

> Only the strike clip contains hit events. On the casting layer, **IK Pass** lets the avatar script adjust hand targets during the swing. The attack section explains the event timing and the hand guard.

PARCEL 1 / 05

Press E at the plant

The fireflies attach by following a target on the right-hand bone. Their hierarchy stays under BeeSwarm.

1. Check the button and range

`FireflyHandAttraction.Update()` uses `CanAttract`: the player must be within **3 m** of the swarm anchor and the swarm must still be at the plant. The same check drives the interaction prompt. A new press marks `attracted` true, assigns the hand target, remembers the starting gap, and sets the attraction animation trigger.

```
currentTarget = beeHandTarget;
remainingOffset = transform.position - beeHandTarget.position;
if (playerAnimator) playerAnimator.SetTrigger("AttractFireflies");
```

2. Close the gap each frame

`BeeHandTarget` is a child of `Hand.R` with local position approximately **(0.031, 0.172, 0.013)**. The target follows the animated bone. During `LateUpdate()` the swarm reads the target's latest world position and moves the saved gap toward zero at **5 m per second**.

```
remainingOffset = Vector3.MoveTowards(
    remainingOffset, Vector3.zero, movementSpeed * Time.deltaTime);
transform.position = currentTarget.position + remainingOffset;
```

At the instant of targeting, adding the original gap to the target gives the swarm's current position. On later frames the gap becomes shorter. Once it is zero, the swarm anchor matches the hand target every frame and `IsAtHand` becomes true. This also works while the player moves or casts.

3. Keep the insects flying

After the three Animators write their local positions, the script eases `formationSize` toward **0.30** and multiplies each model's position by that number. This makes the paths compact around the hand while keeping their individual phase and speed. The world-position follow keeps the swarm out of the hand's parent hierarchy, so hand scale cannot resize it.

> **Two targets on one bone.** The swarm follows `BeeHandTarget` while it flies. The sword later becomes a child of `SwordHandTarget` and aligns its handle to that separate socket.

PARCEL 1 / 06

How the buzzing guides you

Two audio sources use the same looping FireflyBuzz.wav asset. The plant and the hand each have their own volume rules.

Plant audio

`FireflyPlantBuzz` stays at the plant, near world position **(153.562, 17.474, 195.455)**. The script calculates 3D distance and lowers the volume from a maximum of **0.45** to zero across **14 m**. When the swarm leaves, the volume eases to zero and the loop stops. The source is 2D and the script controls how loud it sounds.

Hand audio and detector math

```
float facing = Mathf.Clamp01(Vector3.Dot(player.forward, direction.normalized));
float proximity = 1f - Mathf.Clamp01(distance / detectionRange);
float wantedStrength = facing * proximity;
strength = Mathf.Lerp(strength, wantedStrength,
    1f - Mathf.Exp(-signalSmoothing * Time.deltaTime));
```

The snippet shows the main calculation. The full script flattens the direction on the Y axis and handles a near-zero direction vector. Facing the stone gives a dot product near 1. Sideways or behind gives 0 after clamping. Proximity goes from 1 at the stone to 0 at **65 m**. Both terms must be high for a strong result.

For example, facing straight at the stone from 13 m gives a target signal of **0.80**. The smoothed value controls the slider fill, audio volume **0.022 to 0.68**, and pitch **0.85 to 1.30**. Signal smoothing is **3**. The detector plays only after `IsAtHand` and shuts off as soon as release begins.



Whisper of the Swarm near the bottom and the quest objective at the upper left. Earlier Play Mode capture of the meter and text styling.

The **Whisper of the Swarm** slider is 208 × 7 at a 1920 × 1080 Canvas Scaler reference resolution. A bronze border, gold fill, and dark brown track fit the medieval HUD. The latest objective volume rule is detailed on page 13.

PARCEL 1 / 07

Press E at the stone

The second spell sends the same three animated fireflies from the hand to a point above the rock.

The user-selected `Rock_Medium_01` in front of `Statue_OnRock_Perch_02` is now `MagicStone`. Its world position is about **(196.742, 15.826, 206.817)**. The existing rock mesh, collider, and LODs form the intact stone. `StoneBeeTarget` is a child near local position **(−0.043, 2.164, −0.027)**.

Release conditions and arrival

01 The detector checks for bees at the hand and a player within **3.5 m** of the stone's center. A fresh E press calls `swarm.FlyToStone(stoneBeeTarget)`.

02 The swarm clears `IsAtHand`, sets the two-handed release trigger, and records a new gap to the stone target. The detector meter and hand buzz stop.

03 The anchor closes the new gap while the three Animators keep running. The formation eases toward **2.0** in the saved scene. The script's field initializer is **1.5**, which is the starting value for a new component.

04 On arrival `IsAtStone` becomes true. The detector enables the blue effect, and the mission plays the awakening chime and displays the next objective.

How the stone glows gradually

```
brightness = Mathf.MoveTowards(brightness, 1f, Time.deltaTime / fadeDuration);
// SetGlow also updates the surface tint and emission.
blueLight.intensity = lightBrightness * brightness;
```

`MagicStoneBlueGlow` starts inactive. Its surface is a transparent emissive overlay, paired with a blue point light. `MagicStoneGlow.OnEnable()` takes a runtime material copy and resets brightness to zero. The fade builds across **3 s**, from invisible to the scene's HDR blue emission **(0, 0.35, 1.7)** and a light intensity of **9**.



Play Mode capture from the gradual blue glow check.

PARCEL 1 / 08

Left-click and hit timing

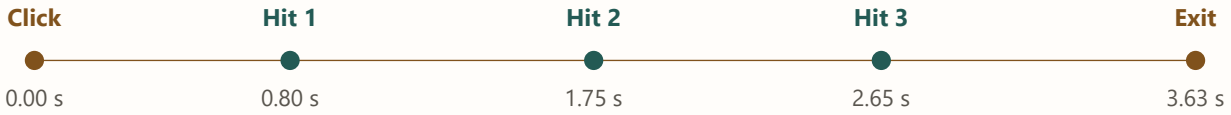
Mouse input starts the swing. Animation events ask the stone to count a hit at three points in the motion.

On each `Player/Attack` press, `PlayerSwordPuzzle.Update()` stops any existing strike coroutine and starts a new one. `Strike()` sets `IsAttacking` and `strikeEvents.striking`, then plays the casting-layer state from normalized time zero.

```
int layer = animator.GetLayerIndex("Firefly Casting");
animator.Play("Firefly Casting.Strike Magic Stone", layer, 0f);
yield return new WaitForSeconds(comboDuration + 0.3f);
```

Left-click works anywhere, including before the rock is awakened and after it breaks. Another click restarts the same clip. The coroutine's flag stays active through the 3.63-second clip plus a 0.30-second margin. Locomotion remains on the base layer.

STRIKE MAGIC STONE / CLIP TIME



Events ask for a hit. The range, facing, and stone state determine whether it counts.

When a hit counts

The imported clip calls `HitStone()` at each marker. The receiver lives on the same GameObject as the Animator. It checks that an attack is active, the stone is intact, the player is within **4.1 m**, and the horizontal forward dot product is greater than **0.5** (roughly within 60°). `CrackableMagicStone.TakeHit()` also checks that the bees have reached the stone.

How the hands stay outside the mesh

`StoneStrikeEvents.OnAnimatorIK()` runs on the casting layer. It resets both hand IK weights, then checks a ray from each upper arm toward its animated hand. If the ray hits the rock, the goal moves to the hit point plus the surface normal times **0.16 m**. A second ray from the player side helps catch poses that start close to the mesh. The Animator solves the corrected arm pose.

To inspect the three markers, select the club FBX and open the Animation tab's Events section. The FBX importer stores normalized times (about 0.2202, 0.4817, 0.7294), while the imported clip reports seconds.

PARCEL 1 / 09

The break, fall, and hover

The third valid hit swaps the rock's visible state and releases the sword to Unity physics.

A three-hit rock

`CrackableMagicStone.TakeHit()` increments `Hits` and plays `StoneImpact.wav` for each valid event. Hits 1 and 2 reveal the two crack meshes. At hit 3, `BreakStone()` sets `IsBroken`, turns off the original rock collider, LOD group and renderers, hides the cracks and blue effect, and disables the detector.

Eight preconfigured rubble bodies become visible and receive outward/upward impulses. Their colliders ignore the player, the sword, and one another. The root and `StoneBeeTarget` remain in the scene, so the fireflies can keep circling. The final hit also plays `StoneBreak.wav`.

From a fixed prop to a floating reward

State	What FloatingSword does
Embedded	The sword Rigidbody starts kinematic, with gravity off and its collider disabled. The authored tilt and child pose show the hilt above the stone.
Fall	`ReleaseFromStone()` enables the collider and gravity, sets the body dynamic, uses continuous collision detection, and adds a small twist.
Landing	The coroutine waits at least 0.35 s, then waits up to 1.5 s more for contact. An upward contact normal above 0.4 counts as a landing.
Hover	Velocity is cleared and the body becomes kinematic. The center is 0.9 m above the contact point, or 0.4 m above the current position without a landing. A 0.1 m sine bob and smoothed rotation make it hover and turn.



The released sword in its hovering state, captured during the original stone-and-sword Play Mode check.

PARCEL 1 / 10

Fitting the sword in the palm

The hand socket and the model's handle point solve two different parts of the alignment.

The user's `Sword15_Frost` model remains a child of `FrostSwordTreasure`. The embedded root's Euler angles are approximately **(0°, 20°, 145°)**. The model has local position **(0.112, 0.268, −0.076)**, local Z rotation **16.388°**, and uniform scale **0.45**. Those child offsets are the authored stone display pose.

What happens on pickup

01 When the hovering sword is within **2.6 m** and the avatar is free from the strike, E calls `FloatingSword.PickUp(swordHandTarget)`.

02 The sword stops hovering, sets `IsHeld`, becomes kinematic, and disables its collider. The root is parented to the animated socket using `SetParent(hand, true)` and then aligned to the socket's local origin and rotation.

03 The visual child's rotation is reset for the grip pose. Its local position becomes the negative of the handle point times the model scale. At the current scale the Y offset is +0.0225, which puts the selected handle point at the socket.

```
transform.SetParent(hand, true);
transform.SetLocalPositionAndRotation(Vector3.zero, Quaternion.identity);
swordVisual.localRotation = Quaternion.identity;
swordVisual.localPosition = -Vector3.Scale(modelGripPoint, swordVisual.localScale);
```

`SwordHandTarget` was fitted to the right palm near local position **(−0.0304, 0.1033, 0.0039)** and local Euler angles **(6.20°, 274.76°, 268.87°)**. The socket aligns the shaft across the fingers. `SwordHandGrip.LateUpdate()` applies 15 recorded finger rotations from the supplied one-handed combo. The arm still follows the Animator. The held blue light drops to **0.4** so the hand is visible.



Close view of the calibrated right-palm grip. Adjust the socket for palm alignment or Model Grip Point for a different sword.

PARCEL 1 / 11

Tap to equip, hold to store

The same E binding supports two distinct presses because the player script checks the sword's current state.

Your input	What the script checks	Result
Fresh E near hovering sword	Within 2.6 m and outside the active attack.	The sword moves into the hand and the mission completes.
Keep holding the pickup press	The press was already used by the pickup branch.	The sword remains equipped.
Release, then press E while held	`holdingToStore` becomes true. The timer counts while E stays down and no attack is active.	The blade stores after 1 second.
Release early or start an attack	The script clears the hold flag and timer.	The blade stays in the hand.

What the inventory actually stores

`storedBlades` is an array of eight GameObject references on `PlayerRoot`. `StoreSword()` scans from slot 0 and uses the first empty slot. It adds one to `storedBladeCount`, clears the sword's held flag through `FloatingSword.Store()`, moves the sword under `StoredBlades`, and makes it inactive. The reference stays in the array. If every slot is filled, the sword stays held.

```
storedBlades[i] = sword.gameObject;
storedBladeCount++;
sword.Store(storage);
```

Stopping Play Mode resets the mission and the array. This prototype stores the sword for the current Play session. The victory sequence starts as soon as pickup succeeds, so players can read it while deciding whether to store the sword.



Earlier Play Mode capture of the victory screen while the sword is still in hand. The current hold prompt says "Hold E again to store the sword."

PARCEL 1 / 12

Mission stages, text, and chimes

One enum in FirstPuzzleMission.cs records which result the puzzle is waiting for.

Stage after the event	Event	Objective and subtitle
`Approach`	Initial state.	The objective is hidden. The plant can already make a faint sound.
`FindFireflies`	Plant buzz is loud, or the player has already attracted the swarm.	Follow the mysterious humming. *Where whispers hum, forgotten steel lies.*
`FollowSwarm`	The swarm reaches the hand.	Follow the bees' humming. *Three wandering lights shall guide your way.* *Let their song unveil what stone conceals.*
`BreakStone`	The swarm reaches the stone.	Break the enchanted stone. *The sleeping stone has awakened.*
`ClaimSword`	The third valid hit breaks the rock.	Press E to claim the sword. *The forgotten blade awaits its keeper.*
`Complete`	The sword is equipped.	Objective hides. *MISSION COMPLETED*, *The First Blade Acquired*, then *A blade is found. A legend begins.*

Why the humming objective now disappears

`UpdateObjectiveVisibility()` checks the active AudioSource's `isPlaying` and smoothed `volume`. The current **Objective Buzz Volume is 0.25**. At the plant, the card appears only when the plant buzz is loud and the swarm is still there. At the hand, the card appears only while the directional detector is loud. Walking away or turning away can hide the card without resetting the stage.

The upper-left card, lower hints, and subtitles use TextMeshPro with EB Garamond and a parchment-and-bronze palette. The subtitle CanvasGroup fades using `MoveTowards` and holds each line for 3 seconds. A newer stage can replace an earlier subtitle. The timers govern presentation, while gameplay flags govern the stage.

The HUD AudioSource plays `StoneAwakens.wav` on stone arrival and `FirstBladeChime.wav` on pickup. Both are short synthesized effects at a HUD source volume of 0.75.

PARCEL 1 / 13

Inspector wiring and assets

In the Hierarchy, select the object in the first column and inspect the corresponding component.

Scene object	Components and assignments
BeeSwarm	**Firefly Hand Attraction** → PlayerRoot, BeeHandTarget, avatar Animator, Interact action, three Fred model transforms. **Firefly Stone Detector** → swarm component, MagicStone, StoneBeeTarget, hand AudioSource, FireflyPlantBuzz source, HUD slider, MagicStoneBlueGlow.
Axe Warrior Avatar	**Animator** → AxeWarriorAnimator and the avatar rig. **Stone Strike Events** → MagicStone and same Animator. **Sword Hand Grip** → FloatingSword, right-hand finger transforms, and their matching closed rotations.
PlayerRoot	**Player Sword Puzzle** → MagicStone, FloatingSword, avatar Animator and strike receiver, Attack and Interact actions, SwordHandTarget, StoredBlades.
MagicStone	**Crackable Magic Stone** → detector, sword, solid MeshCollider, original LOD group and renderers, crack objects, rubble, impact AudioSource and two clips. **Magic Stone Glow** on the glow child → overlay renderer and BlueMagicLight.
FrostSwordTreasure	**Floating Sword** → its Rigidbody and BoxCollider, Sword15_Frost visual, Model Grip Point (0, −0.05, 0), FrostSwordLight.
Whispering Fireflies HUD	**First Puzzle Mission** → detector and player components, objective/hint/subtitle text, subtitle CanvasGroup, AudioSource, awakening and victory chimes. The child slider is Whisper of the Swarm.

Where the files live

- `Assets/GlowKeeper/Scripts/` holds the nine puzzle C# files. Their full source begins on page 16.
- `Assets/GlowKeeper/Animation/Fireflies/` holds the three position clips and controllers. `Animation/AxeWarrior/` holds the supplied FBXs, the avatar controller, and the casting mask.
- `Assets/GlowKeeper/Input/PlayerControls.inputactions` holds both button bindings. `Assets/GlowKeeper/Puzzle/` contains buzzing, impact and break sounds, and the glow materials.
- `Puzzle/Mission/` holds the chimes, font, and its license. `Puzzle/StoneSword/` holds the crack and frost-sword materials.

The original model assets are `Assets/FireFly/source/Firefly_Fred.fbx` and the `Sword15_Frost.prefab` inside `Assets/Blink/Art/Weapons/LowPoly/FreeSwords/Prefabs/`.

PARCEL 1 / 14

Testing the finished puzzle

Open Glowkeeper_Update in Unity and press Play. Follow this sequence to reproduce the checks.

01 At spawn, confirm the objective starts hidden. Move toward the plant until the buzz gets stronger, then step away and return. The top-left card should appear, hide, and reappear.

02 Within 3 m press E. Verify the one-handed animation and the three independent fireflies following your right hand while you move. Turn away from the stone and back toward it. Compare the buzz, the bottom meter, and the top-left objective.

03 At the rock in front of the statue, move within 3.5 m and press E. Confirm the two-handed animation, the meter shutting off, and the chime and gradual blue glow after the fireflies arrive.

04 Face the activated rock within 4.1 m. Left-click once and let the entire clip run. Watch for two crack stages and a break on the third valid hit. The sword should fall and then hover. A click elsewhere still plays the swing.

05 Approach the hovering blade and press E. Check the palm fit while walking, running, turning, and attacking. The victory chime and all three lines should play and the last line should fade away.

06 Release E and hold it again for one second. Check `PlayerRoot → Player Sword Puzzle → Stored Blades`. Slot 0 should contain the sword. Stop Play to reset.

Evidence reviewed for this report

The saved scene, Animator layers, clip events, and Inspector values were read directly in Unity. Prior Play Mode checks drove the same keyboard and mouse actions, repositioning the avatar between locations for repeatable checks. They verified the volume threshold, both spells, three timed hits, gravity and hover, separate pickup and storage presses, and the victory sequence.

The palm-fit test logged **336 samples** across movement and attack, with a reported handle-to-socket gap of **0.000000 m** at the test's displayed precision. This measurement compares the calibrated points during that run. The exact fit can change with a different avatar or sword model.

> **Prototype scope.** This is one scene's mission. Inventory is an in-memory reference array. Hand clearance checks the selected intact rock's collider. Recalibrate the socket and finger pose if you swap the character or blade.

Verification records: `SetupBackup/WhisperingFireflies/MissionPlaycheck.txt`, `HummingVisibilityPlaycheck.txt`, and `AgentScripts/sword-palm-playcheck.txt`. Scene values in this report take precedence over earlier project walkthroughs.

PARCEL 1 / APPENDIX

Appendix · puzzle C# source

These listings come directly from Assets/GlowKeeper/Scripts. Click a filename to jump to its code.

Source file	Responsibility	Lines
FireflyHandAttraction.cs	Moves the anchor from plant to hand to stone.	88
FireflyStoneDetector.cs	Runs the two buzz sources, signal meter, and stone release check.	93
MagicStoneGlow.cs	Fades in the overlay material and blue light on arrival.	47
PlayerSwordPuzzle.cs	Reads attack, pickup, and storage input.	104
StoneStrikeEvents.cs	Handles the animation events and the two hand IK goals.	60
CrackableMagicStone.cs	Counts hits and switches the intact rock to rubble.	55
FloatingSword.cs	Drops, hovers, attaches, and stores the sword object.	98
SwordHandGrip.cs	Applies the calibrated finger pose while the sword is held.	16
FirstPuzzleMission.cs	Updates stage, text, objective visibility, and chimes.	133

Reading the listings

A public field at the top of a script normally corresponds to an Inspector value or reference. The scene can override an initializer in the script. For example, the stone formation size starts at 1.5 on a new component, while the assigned scene instance uses 2.0.

`Update()` runs each frame. `LateUpdate()` runs after the normal update work and is used here to follow animated targets, apply the grip pose, and update the mission UI. A coroutine uses `yield return` to resume later, which makes falling, timing an attack, and fading text easy to read in order.

Some long source lines wrap for print. The continuation row carries no line number. The code sections earlier in the report are short teaching excerpts.

C# SOURCE / 1 OF 2

FireflyHandAttraction.cs

Lines 1–45 · Moves the anchor from plant to hand to stone.

```
using UnityEngine;
using UnityEngine.InputSystem;

public class FireflyHandAttraction : MonoBehaviour
{
    [Header("Scene references")]
    public Transform player;
    public Transform beeHandTarget;
    public Animator playerAnimator;
    public InputActionReference interactAction;
    [Tooltip("The three Firefly_Fred models inside the animated holders.")]
    public Transform[] fireflies = new Transform[3];

    [Header("Settings")]
    [Min(0.1f)] public float interactionDistance = 3f;
    [Min(0.1f)] public float movementSpeed = 5f;
    [Range(0.1f, 1f)] public float handFormationSize = 0.3f;
    [Min(0.1f)] public float stoneFormationSize = 1.5f;

    public bool IsAtHand { get; private set; }
    public bool IsAtStone { get; private set; }
    public bool HasLeftPlant => attracted;
    public bool CanAttract => !attracted && player &&
        Vector3.Distance(player.position, transform.position) <= interactionDistance;

    private bool attracted;
    private Vector3 remainingOffset;
    private float formationSize = 1f;
    private Transform currentTarget;
    private bool goingToStone;

    private void OnEnable()
    {
        if (interactAction) interactAction.action.Enable();
    }

    private void OnDisable()
    {
        if (interactAction) interactAction.action.Disable();
    }

    private void Update()
    {
        if (!player || !beeHandTarget || !interactAction) return;

```

C# SOURCE / 2 OF 2

FireflyHandAttraction.cs

Lines 46–88 · Moves the anchor from plant to hand to stone.

```
46          if (CanAttract && interactAction.action.WasPressedThisFrame())
47          {
48              attracted = true; // E works only once for this swarm.
49              currentTarget = beeHandTarget;
50              remainingOffset = transform.position - beeHandTarget.position;
51              if (playerAnimator) playerAnimator.SetTrigger("AttractFireflies");
52          }
53      }
54
55      public void FlyToStone(Transform stoneTarget)
56      {
57          if (!IsAtHand || !stoneTarget) return;
58          // Play the two-handed spell as the bees leave the hand.
59          if (playerAnimator) playerAnimator.SetTrigger("ReleaseFireflies");
60          IsAtHand = false;
61          goingToStone = true;
62          currentTarget = stoneTarget;
63          // Remember the current gap so changing destinations never teleports the bees.
64          remainingOffset = transform.position - currentTarget.position;
65      }
66
67      private void LateUpdate()
68      {
69          if (!attracted || !currentTarget) return;
70
71          // Close the gap while following the animated hand. Never change parents or scales.
72          remainingOffset = Vector3.MoveTowards(remainingOffset, Vector3.zero, movementSpeed *
    Time.deltaTime);
73          transform.position = currentTarget.position + remainingOffset;
74          float desiredSize = goingToStone ? stoneFormationSize : handFormationSize;
75          formationSize = Mathf.MoveTowards(formationSize, desiredSize, Time.deltaTime);
76
77          if (remainingOffset == Vector3.zero)
78          {
79              IsAtHand = !goingToStone;
80              IsAtStone = goingToStone;
81          }
82
83          // Each Animator has just updated its local flight position. Make that same
84          // flight smaller around the hand, preserving its phase, speed and model size.
85          foreach (Transform firefly in fireflies)
86              if (firefly) firefly.localPosition *= formationSize;
87      }
88  }
```

C# SOURCE / 1 OF 2

FireflyStoneDetector.cs

Lines 1–46 · Runs the two buzz sources, signal meter, and stone release check.

```
using UnityEngine;
using UnityEngine.UI;

public class FireflyStoneDetector : MonoBehaviour
{
    [Header("Drag in the scene objects")]
    public FireflyHandAttraction swarm;
    public Transform magicStone;
    public Transform stoneBeeTarget;
    public AudioSource buzzing;
    public Slider detectorMeter;
    public GameObject blueGlow;

    [Header("Buzzing at the plant")]
    public AudioSource plantBuzz;
    [Min(0.1f)] public float plantBuzzRange = 14f;
    [Range(0f, 1f)] public float plantMaxVolume = 0.45f;

    [Header("Detector settings")]
    [Min(0.1f)] public float detectionRange = 65f;
    [Min(0.1f)] public float closeActivationDistance = 3.5f;
    [Min(0.1f)] public float signalSmoothing = 3f;
    [Range(0f, 1f)] public float minVolume = 0.06f;
    [Range(0f, 1f)] public float maxVolume = 0.6f;
    [Range(0.1f, 3f)] public float minPitch = 0.85f;
    [Range(0.1f, 3f)] public float maxPitch = 1.3f;

    public bool CanAwakenStone => swarm.IsAtHand && StoneDistance <= closeActivationDistance;

    private float strength;
    private float StoneDistance => Vector3.Distance(swarm.player.position, magicStone.position);

    private void Update()
    {
        if (!swarm || !magicStone || !stoneBeeTarget) return;

        UpdatePlantBuzz();

        if (CanAwakenStone && swarm.interactAction.action.WasPressedThisFrame())
            swarm.FlyToStone(stoneBeeTarget);

        // The detector works only while the bees are at the player's hand.
        bool searching = swarm.IsAtHand;
        if (detectorMeter) detectorMeter.gameObject.SetActive(searching);
        if (blueGlow && swarm.IsAtStone) blueGlow.SetActive(true);

```

C# SOURCE / 2 OF 2

FireflyStoneDetector.cs

Lines 47–93 · Runs the two buzz sources, signal meter, and stone release check.

```
47          if (!searching)
48          {
49              strength = 0f;
50              if (buzzing && buzzing.isPlaying) buzzing.Stop();
51              return;
52          }
53
54          Vector3 direction = magicStone.position - swarm.player.position;
55          direction.y = 0f; // Look left/right, not up/down at the ground.
56          float facing = direction.sqrMagnitude < 0.001f ? 1f :
57              Mathf.Clamp01(Vector3.Dot(swarm.player.forward, direction.normalized));
58          float proximity = 1f - Mathf.Clamp01(StoneDistance / detectionRange);
59          float wantedStrength = facing * proximity;
60
61          // Smooth one signal, then use it for both sound and the bar.
62          strength = Mathf.Lerp(strength, wantedStrength, 1f - Mathf.Exp(-signalSmoothing *
    Time.deltaTime));
63          if (detectorMeter) detectorMeter.value = strength;
64          if (buzzing)
65          {
66              buzzing.volume = Mathf.Lerp(minVolume, maxVolume, strength);
67              buzzing.pitch = Mathf.Lerp(minPitch, maxPitch, strength);
68              if (!buzzing.isPlaying) buzzing.Play();
69          }
70      }
71
72      private void UpdatePlantBuzz()
73      {
74          if (!plantBuzz || !swarm.player) return;
75
76          float plantDistance = Vector3.Distance(swarm.player.position, plantBuzz.transform.position);
77          float closeness = 1f - Mathf.Clamp01(plantDistance / plantBuzzRange);
78          float wantedVolume = swarm.HasLeftPlant ? 0f : closeness * plantMaxVolume;
79          plantBuzz.volume = Mathf.Lerp(plantBuzz.volume, wantedVolume, 1f - Mathf.Exp(-4f *
    Time.deltaTime));
80          if (plantBuzz.volume > 0.001f)
81          {
82              if (!plantBuzz.isPlaying) plantBuzz.Play();
83          }
84          else if (plantBuzz.isPlaying) plantBuzz.Stop();
85      }
86
87      private void OnDisable()
88      {
89          if (buzzing) buzzing.Stop();
90          if (plantBuzz) plantBuzz.Stop();
91          if (detectorMeter) detectorMeter.gameObject.SetActive(false);
92      }
93  }
```

C# SOURCE / 1 OF 1

MagicStoneGlow.cs

Lines 1–47 · Fades in the overlay material and blue light on arrival.

```
using UnityEngine;

public class MagicStoneGlow : MonoBehaviour
{
    public Renderer stoneSurface;
    public Light blueLight;
    [Min(0.1f)] public float fadeDuration = 3f;
    [ColorUsage(false, true)] public Color glowColor = new Color(0f, 0.65f, 3f);
    [Min(0f)] public float lightBrightness = 9f;

    private Material glowMaterial;
    private float brightness;

    private void OnEnable()
    {
        // This object is enabled only when the bees reach the stone.
        if (!glowMaterial && stoneSurface)
        {
            glowMaterial = stoneSurface.material;
            glowMaterial.EnableKeyword("_EMISSION");
        }
        brightness = 0f;
        SetGlow();
    }

    private void Update()
    {
        // Take a few seconds to go from no glow to full magical blue.
        brightness = Mathf.MoveTowards(brightness, 1f, Time.deltaTime / fadeDuration);
        SetGlow();
    }

    private void SetGlow()
    {
        if (glowMaterial)
        {
            glowMaterial.SetColor("_BaseColor", new Color(0.02f, 0.25f, 1f, brightness * 0.8f));
            glowMaterial.SetColor("_EmissionColor", glowColor * brightness);
        }
        if (blueLight) blueLight.intensity = lightBrightness * brightness;
    }

    private void OnDestroy()
    {
        if (glowMaterial) Destroy(glowMaterial);
    }
}
```

C# SOURCE / 1 OF 2

PlayerSwordPuzzle.cs

Lines 1–54 · Reads attack, pickup, and storage input.

```
using System.Collections;
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerSwordPuzzle : MonoBehaviour
{
    public CrackableMagicStone stone;
    public FloatingSword sword;
    public StoneStrikeEvents strikeEvents;
    public Animator animator;
    public InputActionReference attackAction;
    public InputActionReference interactAction;
    public Transform swordHandTarget;
    public Transform storage;
    public float attackRange = 4.1f;
    public float pickupRange = 2.6f;
    public float comboDuration = 3.633333f;
    public float storeHoldTime = 1f;
    public GameObject[] storedBlades = new GameObject[8];
    public int storedBladeCount;
    public bool IsAttacking { get; private set; }
    public bool CanClaimSword => !IsAttacking && sword && sword.IsHovering &&
        Vector3.Distance(transform.position, sword.transform.position) <= pickupRange;

    private float heldTime;
    private bool holdingToStore;
    private Coroutine strikeRoutine;

    private void OnEnable()
    {
        if (attackAction) attackAction.action.Enable();
    }

    private void Update()
    {
        // Left click always attacks in the direction the player is facing.
        // Another click restarts the combo instead of being ignored.
        if (attackAction.action.WasPressedThisFrame())
        {
            if (strikeRoutine != null) StopCoroutine(strikeRoutine);
            strikeRoutine = StartCoroutine(Strike());
        }
        if (sword && interactAction) UpdateSwordInput();
    }

    private void UpdateSwordInput()
    {
        // A new press either picks up the sword or begins a storage hold.
        if (!IsAttacking && interactAction.action.WasPressedThisFrame())
        {
            if (CanClaimSword) sword.PickUp(swordHandTarget);
            else if (sword.IsHeld) holdingToStore = true;
        }

```

C# SOURCE / 2 OF 2

PlayerSwordPuzzle.cs

Lines 55–104 · Reads attack, pickup, and storage input.

```
        if (IsAttacking || !interactAction.action.IsPressed())
        {
            holdingToStore = false;
            heldTime = 0f;
        }
        if (!holdingToStore) return;

        heldTime += Time.deltaTime;
        if (heldTime < storeHoldTime) return;
        StoreSword();
        holdingToStore = false;
        heldTime = 0f;
    }

    private IEnumerator Strike()
    {
        IsAttacking = true;
        strikeEvents.striking = true;
        int layer = animator.GetLayerIndex("Firefly Casting");
        animator.Play("Firefly Casting.Strike Magic Stone", layer, 0f);
        yield return new WaitForSeconds(comboDuration + 0.3f);
        strikeEvents.striking = false;
        IsAttacking = false;
        strikeRoutine = null;
    }

    private void StoreSword()
    {
        for (int i = 0; i < storedBlades.Length; i++)
        {
            if (storedBlades[i]) continue;
            storedBlades[i] = sword.gameObject;
            storedBladeCount++;
            sword.Store(storage);
            return;
        }
    }

    private void OnDisable()
    {
        if (attackAction) attackAction.action.Disable();
        if (IsAttacking)
        {
            StopAllCoroutines();
            strikeEvents.striking = false;
            IsAttacking = false;
            strikeRoutine = null;
        }
    }
}
```

C# SOURCE / 1 OF 1

StoneStrikeEvents.cs

Lines 1–60 · Handles the animation events and the two hand IK goals.

```
using UnityEngine;

// Put this on the avatar, because its Animator sends the hit events here.
public class StoneStrikeEvents : MonoBehaviour
{
    public CrackableMagicStone stone;
    public Animator animator;
    public bool striking;
    public float handClearance = 0.16f;

    public void HitStone()
    {
        if (!striking || stone.IsBroken) return;
        var player = GetComponentInParent<PlayerSwordPuzzle>();
        Vector3 direction = stone.transform.position - player.transform.position;
        direction.y = 0f;
        bool near = Vector3.Distance(player.transform.position, stone.transform.position) <=
    player.attackRange;
        bool facing = Vector3.Dot(player.transform.forward, direction.normalized) > 0.5f;
        // The animation works anywhere. Only a nearby, correctly aimed hit damages the stone.
        if (near && facing) stone.TakeHit();
    }

    private void OnAnimatorIK(int layerIndex)
    {
        if (layerIndex != animator.GetLayerIndex("Firefly Casting")) return;
        animator.SetIKPositionWeight(AvatarIKGoal.RightHand, 0f);
        animator.SetIKPositionWeight(AvatarIKGoal.LeftHand, 0f);
        if (!striking || stone.IsBroken || !stone.solidCollider.enabled) return;
        KeepHandOutside(AvatarIKGoal.RightHand, HumanBodyBones.RightUpperArm);
        KeepHandOutside(AvatarIKGoal.LeftHand, HumanBodyBones.LeftUpperArm);
    }

    private void KeepHandOutside(AvatarIKGoal goal, HumanBodyBones arm)
    {
        Vector3 shoulder = animator.GetBoneTransform(arm).position;
        Vector3 hand = animator.GetIKPosition(goal);
        Vector3 direction = hand - shoulder;
        Vector3 target = hand;
        RaycastHit hit;
        // Stop the hand just outside the rock, even if the clip swings through it.
        if (stone.solidCollider.Raycast(new Ray(shoulder, direction.normalized), out hit,
    direction.magnitude + handClearance))
            target = hit.point + hit.normal * handClearance;
        else
        {
            // Also protect hands when a swing starts very close to the rock.
            Vector3 outward = animator.transform.position - stone.solidCollider.bounds.center;
            outward.y = 0f;
            if (outward.sqrMagnitude < 0.001f) outward = -animator.transform.forward;
            outward.Normalize();
            float reach = stone.solidCollider.bounds.size.magnitude + 1f;
            if (stone.solidCollider.Raycast(new Ray(hand + outward * reach, -outward), out hit, reach +
    handClearance))
                target = hit.point + hit.normal * handClearance;
        }
        if (target != hand)
        {
            animator.SetIKPositionWeight(goal, 1f);
            animator.SetIKPosition(goal, target);
        }
    }
}
```

C# SOURCE / 1 OF 1

CrackableMagicStone.cs

Lines 1–55 · Counts hits and switches the intact rock to rubble.

```
1  using UnityEngine;
2
3  public class CrackableMagicStone : MonoBehaviour
4  {
5      public FireflyStoneDetector detector;
6      public FloatingSword sword;
7      public Collider solidCollider;
8      public LODGroup stoneLOD;
9      public Renderer[] stoneRenderers;
10     public GameObject[] crackStages;
11     public GameObject rubble;
12     public AudioSource impactAudio;
13     public AudioClip hitSound;
14     public AudioClip breakSound;
15     public int hitsToBreak = 3;
16     public bool IsBroken { get; private set; }
17     public int Hits { get; private set; }
18
19     public void TakeHit()
20     {
21         if (IsBroken || !detector.swarm.IsAtStone) return;
22         Hits++;
23         impactAudio.PlayOneShot(hitSound);
24         if (Hits >= hitsToBreak) BreakStone();
25         else if (Hits - 1 < crackStages.Length) crackStages[Hits - 1].SetActive(true);
26     }
27
28     private void BreakStone()
29     {
30         IsBroken = true;
31         solidCollider.enabled = false;
32         if (stoneLOD) stoneLOD.enabled = false;
33         foreach (Renderer surface in stoneRenderers) surface.enabled = false;
34         foreach (GameObject cracks in crackStages) cracks.SetActive(false);
35         // Detection is finished. Leave the swarm and its target alive.
36         detector.enabled = false;
37         detector.blueGlow.SetActive(false);
38         rubble.SetActive(true);
39         sword.ReleaseFromStone();
40         var pieces = rubble.GetComponentsInChildren<Rigidbody>();
41         var playerCollider = detector.swarm.player.GetComponent<CharacterController>();
42         foreach (Rigidbody piece in pieces)
43         {
44             var collider = piece.GetComponent<Collider>();
45             Physics.IgnoreCollision(collider, playerCollider);
46             Physics.IgnoreCollision(collider, sword.swordCollider);
47             foreach (Rigidbody other in pieces)
48                 if (other != piece) Physics.IgnoreCollision(collider, other.GetComponent<Collider>());
49             piece.isKinematic = false;
50             Vector3 outward = (piece.position - transform.position).normalized;
51             piece.AddForce((outward + Vector3.up * 0.5f) * 1.5f, ForceMode.Impulse);
52         }
53         impactAudio.PlayOneShot(breakSound);
54     }
55 }
```

C# SOURCE / 1 OF 2

FloatingSword.cs

Lines 1–42 · Drops, hovers, attaches, and stores the sword object.

```
using System.Collections;
using UnityEngine;

public class FloatingSword : MonoBehaviour
{
    public Rigidbody body;
    public Collider swordCollider;
    public float hoverHeight = 0.9f;
    public float bobHeight = 0.1f;
    [Header("Hand grip")]
    public Transform swordVisual;
    [Tooltip("Center of the handle in the sword model's own coordinates.")]
    public Vector3 modelGripPoint = new Vector3(0f, -0.05f, 0f);
    public Light swordLight;
    public float heldLightIntensity = 0.4f;
    public bool IsHovering { get; private set; }
    public bool IsHeld { get; private set; }

    private bool landed;
    private Vector3 landingPoint;
    private Vector3 hoverCenter;

    public void ReleaseFromStone()
    {
        // Real gravity drops the sword before we start the magic hovering.
        body.isKinematic = false;
        body.useGravity = true;
        swordCollider.enabled = true;
        body.collisionDetectionMode = CollisionDetectionMode.ContinuousDynamic;
        body.AddTorque(transform.forward * 0.35f, ForceMode.Impulse);
        StartCoroutine(FallThenHover());
    }

    private void OnCollisionEnter(Collision collision)
    {
        if (collision.GetContact(0).normal.y > 0.4f)
        {
            landed = true;
            landingPoint = collision.GetContact(0).point;
        }
    }

```

C# SOURCE / 2 OF 2

FloatingSword.cs

Lines 43–98 · Drops, hovers, attaches, and stores the sword object.

```
43      private IEnumerator FallThenHover()
44      {
45          yield return new WaitForSeconds(0.35f);
46          float waiting = 0f;
47          while (!landed && waiting < 1.5f)
48          {
49              waiting += Time.deltaTime;
50              yield return null;
51          }
52          body.linearVelocity = Vector3.zero;
53          body.angularVelocity = Vector3.zero;
54          body.collisionDetectionMode = CollisionDetectionMode.ContinuousSpeculative;
55          body.isKinematic = true;
56          hoverCenter = transform.position + Vector3.up * 0.4f;
57          if (landed) hoverCenter.y = landingPoint.y + hoverHeight;
58          IsHovering = true;
59      }
60
61      private void Update()
62      {
63          if (!IsHovering) return;
64          Vector3 target = hoverCenter + Vector3.up * Mathf.Sin(Time.time * 2f) * bobHeight;
65          float smooth = 1f - Mathf.Exp(-5f * Time.deltaTime);
66          transform.position = Vector3.Lerp(transform.position, target, smooth);
67          transform.rotation = Quaternion.Slerp(transform.rotation, Quaternion.Euler(0f, Time.time * 35f,
    0f), smooth);
68      }
69
70      public void PickUp(Transform hand)
71      {
72          StopAllCoroutines();
73          IsHovering = false;
74          IsHeld = true;
75          body.isKinematic = true;
76          swordCollider.enabled = false;
77          transform.SetParent(hand, true); // Keep the sword's world size.
78          transform.SetLocalPositionAndRotation(Vector3.zero, Quaternion.identity);
79          // Stone-placement offsets must not leave the handle floating beside the palm.
80          if (swordVisual)
81          {
82              swordVisual.localRotation = Quaternion.identity;
83              swordVisual.localPosition = -Vector3.Scale(modelGripPoint, swordVisual.localScale);
84          }
85          if (swordLight)
86          {
87              swordLight.transform.localPosition = Vector3.zero;
88              swordLight.intensity = heldLightIntensity;
89          }
90      }
91
92      public void Store(Transform storage)
93      {
94          IsHeld = false;
95          transform.SetParent(storage, true);
96          gameObject.SetActive(false); // The inventory array still remembers it.
97      }
98  }
```

C# SOURCE / 1 OF 1

SwordHandGrip.cs

Lines 1–16 · Applies the calibrated finger pose while the sword is held.

```
1  using UnityEngine;
2
3  // Close only the right-hand fingers while this sword is held.
4  public class SwordHandGrip : MonoBehaviour
5  {
6      public FloatingSword sword;
7      public Transform[] fingers;
8      public Quaternion[] closedRotations;
9
10     private void LateUpdate()
11     {
12         if (!sword || !sword.IsHeld) return;
13         for (int i = 0; i < fingers.Length; i++)
14             fingers[i].localRotation = closedRotations[i];
15     }
16 }
```

C# SOURCE / 1 OF 3

FirstPuzzleMission.cs

Lines 1–49 · Updates stage, text, objective visibility, and chimes.

```
using System.Collections;
using TMPro;
using UnityEngine;

public class FirstPuzzleMission : MonoBehaviour
{
    public FireflyStoneDetector detector;
    public PlayerSwordPuzzle player;

    public TextMeshProUGUI objective;
    public TextMeshProUGUI prompt;
    public TextMeshProUGUI subtitle;
    public CanvasGroup subtitleGroup;
    public AudioSource chimes;
    public AudioClip awakeningChime;
    public AudioClip victoryChime;
    [Range(0f, 1f)] public float objectiveBuzzVolume = 0.25f;

    private enum Stage { Approach, FindFireflies, FollowSwarm, BreakStone, ClaimSword, Complete }
    private Stage stage;
    private Coroutine subtitleRoutine;

    private void LateUpdate()
    {
        CheckProgress();
        UpdateObjectiveVisibility();
        UpdatePrompt();
    }

    // Read the existing puzzle's results. This script never handles input.
    private void CheckProgress()
    {
        if (stage == Stage.Approach && (IsLoud(detector.plantBuzz) || detector.swarm.HasLeftPlant))
        {
            stage = Stage.FindFireflies;
            objective.text = "Follow the mysterious humming.";
            if (!detector.swarm.HasLeftPlant)
                Say("Where whispers hum, forgotten steel lies.");
        }
        else if (stage == Stage.FindFireflies && detector.swarm.IsAtHand)
        {
            stage = Stage.FollowSwarm;
            objective.text = "Follow the bees' humming.";
            Say("Three wandering lights shall guide your way.",
                "Let their song unveil what stone conceals.");
        }
        else if (stage == Stage.FollowSwarm && detector.swarm.IsAtStone)
        {
            stage = Stage.BreakStone;
```

C# SOURCE / 2 OF 3

FirstPuzzleMission.cs

Lines 50–84 · Updates stage, text, objective visibility, and chimes.

```
            objective.text = "Break the enchanted stone.";
            chimes.PlayOneShot(awakeningChime);
            Say("The sleeping stone has awakened.");
        }
        else if (stage == Stage.BreakStone && player.stone.IsBroken)
        {
            stage = Stage.ClaimSword;
            objective.text = "Press E to claim the sword.";
            Say("The forgotten blade awaits its keeper.");
        }
        else if (stage == Stage.ClaimSword && player.sword.IsHeld)
        {
            stage = Stage.Complete;
            chimes.PlayOneShot(victoryChime);
            Say("<color=#E8BD73>MISSION COMPLETED</color>", "The First Blade Acquired",
                "A blade is found. A legend begins.");
        }
    }

    private void UpdateObjectiveVisibility()
    {
        bool show = stage == Stage.BreakStone || stage == Stage.ClaimSword;
        if (stage == Stage.FindFireflies)
            show = !detector.swarm.HasLeftPlant && IsLoud(detector.plantBuzz);
        else if (stage == Stage.FollowSwarm)
            show = detector.swarm.IsAtHand && IsLoud(detector.buzzing);

        objective.transform.parent.gameObject.SetActive(show);
    }

    private bool IsLoud(AudioSource buzz)
    {
        return buzz && buzz.isPlaying && buzz.volume >= objectiveBuzzVolume;
    }

```

C# SOURCE / 3 OF 3

FirstPuzzleMission.cs

Lines 85–133 · Updates stage, text, objective visibility, and chimes.

```
85      private void UpdatePrompt()
86      {
87          string message = "";
88          if (detector.swarm.CanAttract)
89              message = "Press E to attract the fireflies.";
90          else if (detector.CanAwakenStone)
91              message = "Press E to awaken the stone.";
92          else if (stage == Stage.BreakStone && !player.IsAttacking &&
93              Vector3.Distance(player.transform.position, player.stone.transform.position) <=
    player.attackRange)
94              message = "Face the stone and left click to strike.";
95          else if (player.CanClaimSword)
96              message = "Press E to claim the sword.";
97          else if (player.sword.IsHeld)
98              message = player.storedBladeCount < player.storedBlades.Length
99                  ? "Hold E again to store the sword." : "Blade storage is full.";
100
101         prompt.text = message;
102         prompt.transform.parent.gameObject.SetActive(message != "");
103     }
104
105     private void Say(params string[] lines)
106     {
107         if (subtitleRoutine != null) StopCoroutine(subtitleRoutine);
108         subtitleRoutine = StartCoroutine(ShowSubtitles(lines));
109     }
110
111     // Time controls only the text fading. Gameplay decides the mission stage.
112     private IEnumerator ShowSubtitles(string[] lines)
113     {
114         subtitleGroup.alpha = 0f;
115         foreach (string line in lines)
116         {
117             subtitle.text = line;
118             yield return FadeTo(1f);
119             yield return new WaitForSeconds(3f);
120             yield return FadeTo(0f);
121         }
122         subtitleRoutine = null;
123     }
124
125     private IEnumerator FadeTo(float alpha)
126     {
127         while (subtitleGroup.alpha != alpha)
128         {
129             subtitleGroup.alpha = Mathf.MoveTowards(subtitleGroup.alpha, alpha, 3f * Time.deltaTime);
130             yield return null;
131         }
132     }
133 }
```